

TP : Développement d'une API REST avec Spring Boot

Objectif : Réaliser une application Spring Boot permettant de gérer des étudiants et leurs cours, en exposant une API REST complète avec persistance en base de données.

Partie 1 : Initialisation du projet

Travail demandé

1. Créer un projet Spring Boot avec :
 - Spring Web
 - Spring Data JPA
 - H2 Database | Postgres | MySQL ...
 - Validation
2. Vérifier que l'application démarre avec :

```
@SpringBootApplication
public class TpApplication {
    public static void main(String[] args) {
        SpringApplication.run(TpApplication.class, args);
    }
}
```

Partie 2 : Modélisation des entités

Entité Student

Créer une classe Student :

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotNull
    @Size(min = 2, max = 50)
    private String name;

    @Email
    private String email;
}
```

Entité Course

```
@Entity
public class Course {
```

```

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
}

```

Relation

- Un étudiant peut avoir plusieurs cours

```

@OneToMany
private List<Course> courses;

```

Partie 3 : Repository

Créer les interfaces :

```

public interface StudentRepository extends JpaRepository<Student, Long> {
    List<Student> findByName(String name);
}

```

```

public interface CourseRepository extends JpaRepository<Course, Long> {
}

```

Partie 4 : Controller REST

Création d'un StudentController

```

@RestController
@RequestMapping("/students")
public class StudentController {

    @Autowired
    private StudentRepository studentRepository;

    @GetMapping
    public List<Student> getAll() {
        return studentRepository.findAll();
    }

    @PostMapping
    public Student create(@RequestBody @Valid Student student) {
        return studentRepository.save(student);
    }

    @GetMapping("/{id}")
    public Student getById(@PathVariable Long id) {
        return studentRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Student not found"));
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) {
        studentRepository.deleteById(id);
    }
}

```

Partie 5 : Gestion des exceptions

Créer une classe globale :

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<String> handleException(RuntimeException ex) {
        return ResponseEntity.badRequest().body(ex.getMessage());
    }
}
```

Partie 6 : Test avec Postman

Endpoints à tester

Méthode	URL	Description
GET	/students	Liste des étudiants
POST	/students	Ajouter un étudiant
GET	/students/{id}	Récupérer un étudiant
DELETE	/students/{id}	Supprimer un étudiant

Partie 7 : Améliorations (Travail supplémentaire ★)

À faire par les étudiants :

- Ajouter :
 - @PutMapping pour modifier un étudiant
- Créer un DTO :

```
public class StudentDTO {
    private String name;
    private String email;
}
```

- Ajouter pagination :

```
Page<Student> findAll(Pageable pageable);
```

- Ajouter une recherche :

```
GET /students/search?name=Ali
```

Partie 8 : Questions de réflexion

- Quelle est la différence entre :

- @RestController et @Controller ?
- 8. Pourquoi utiliser un DTO ?
- 9. Quel est le rôle de @Transactional ?
- 10. Quelle est la différence entre findById() et getByld() ?
- 11. Pourquoi utiliser la validation (@Valid) ?

Bonus (Mini-Projet)

Transformer le TP en système de gestion :

- Étudiants
- Cours
- Inscriptions (ManyToMany)

**Objectifs pédagogiques atteints ✓ Comprendre Spring Boot ✓ Créer une API REST
✓ Utiliser JPA et Hibernate ✓ Implémenter CRUD ✓ Gérer les exceptions ✓ Valider
les données**